SOFTWARE ENGINEERING LAB · SEMESTER 5

Lab 5: Static Code Analysis

School Bus Transport Register — `school_transport.py`

Analysis with Pylint, Bandit and Flake8

Student ID: PES2UG24CS212 · Date: 2026-10-14

1. Objective

Analyse the school bus transport register using three static-analysis tools — **Pylint** (code quality / conventions), **Bandit** (security) and **Flake8** (PEP 8 style) — document the findings, fix at least four genuine issues while preserving the program's behaviour, and reflect on the role and limits of static analysis. Every finding and snippet below comes from the actual project files and the saved tool reports; nothing is invented.

2. Tools Used

Tool	Version	Purpose
Pylint	4.1.3	Correctness, conventions, unused code, naming, docstrings (scored 0–10)
Bandit	1.9.4	Security issues (injection, shell use, asserts) mapped to CWEs
Flake8	7.4.1	PEP 8 style (line length, whitespace, bare except)
Python	3.14.7	Runtime (program executed before and after the fixes)

Environment: A dedicated virtual environment (`.venv`) was used and was deliberately excluded from this report and the repository.

3. Summary of the Original Code Issues

The program stores riders in an in-memory SQLite table and enrols three riders, checks the R1 fee total, exports a roster file, emits a dispatch alert and prints the R2 roster. It runs correctly, but its original `main()` even executes on import (`main()` at module level) and the code has issues at every layer:

- **Security:** an `INSERT` built with an f-string (SQL injection) and a `subprocess.run(..., shell=True)` dispatch call (OS command injection) — both exploitable via a crafted route or name.
- **Code quality:** a bare `except:` that hides failures, an unused `driver` argument, and manual `open()` / `close()` without a context manager or encoding.
- **Style / PEP 8:** camelCase function names, missing module/function docstrings, over-long lines and missing whitespace after commas.

Original tool verdicts

Pylint scored the file **4.00/10**; Bandit reported **1 High, 1 Medium, 2 Low**; Flake8 reported **9** style findings. Representative original Pylint output:

```
* Module school_transport_original
school_transport_original.py:5:0: C0301: Line too long (110/100)
    (line-too-long)
school_transport_original.py:9:0: C0301: Line too long (116/100)
    (line-too-long)
school_transport_original.py:1:0: C0114: Missing module docstring
    (missing-module-docstring)
school_transport_original.py:8:0: C0116: Missing function or method
    docstring (missing-function-docstring)
school_transport_original.py:8:0: C0103: Function name "enrollRider"
    doesn't conform to snake_case naming style (invalid-name)
school_transport_original.py:13:0: C0116: Missing function or method
    docstring (missing-function-docstring)
school_transport_original.py:13:0: C0103: Function name "routeRoster"
    doesn't conform to snake_case naming style (invalid-name)
school_transport_original.py:13:23: W0613: Unused argument 'driver'
    (unused-argument)
school_transport_original.py:17:0: C0116: Missing function or method
    docstring (missing-function-docstring)
school_transport_original.py:17:0: C0103: Function name "exportRoster"
    doesn't conform to snake_case naming style (invalid-name)
school_transport_original.py:18:10: W1514: Using open without explicitly
    specifying an encoding (unspecified-encoding)
school_transport_original.py:18:10: R1732: Consider using 'with' for
    resource-allocating operations (consider-using-with)
school_transport_original.py:23:0: C0116: Missing function or method
    docstring (missing-function-docstring)
school_transport_original.py:23:0: C0103: Function name "dispatchAlert"
    doesn't conform to snake_case naming style (invalid-name)
school_transport_original.py:27:0: C0116: Missing function or method
    docstring (missing-function-docstring)
school_transport_original.py:27:0: C0103: Function name "feeCollected"
    doesn't conform to snake_case naming style (invalid-name)
school_transport_original.py:30:4: W0702: No exception type(s) specified
    (bare-except)
school_transport_original.py:34:0: C0116: Missing function or method
    docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 4.00/10
```

4. Issue Analysis Table

Issue	Type/Tool	Line(s)	Description	Fix Approach	Status
SQL injection via f-string	**Security** — Bandit **B608** (Medium)	9	`enrollRider` built the `INSERT` by interpolating `student`, `route`, `stop`, `fee` directly into the SQL string. Any value containing a quote could break out and inject SQL (CWE-89).	Replaced with a parameterised query using `?` placeholders: `conn.execute("INSERT ... VALUES (?, ?, ?, ?)", (student, route, stop, fee))`. Data is bound, never parsed as SQL.	**Fixed**
Command injection via `shell=True`	**Security** — Bandit **B602** (High), CWE-78	24	`dispatchAlert` ran `subprocess.run("echo dispatching bus " + route, shell=True)`. The untrusted `route` was passed to a real shell, so a value like `R1; rm -rf /` would execute arbitrary commands.	Removed `shell=True` and switched to an argument list: `subprocess.run(["echo", "dispatching bus", route], check=False)`. No shell is spawned, so metacharacters in `route` are inert.	**Fixed**
Overly broad exception handler	**Code quality** — Pylint **W0702** / Flake8 **E722**	30	`feeCollected` used a bare `except:` that swallowed every exception (including `KeyboardInterrupt`/`SystemExit`) and silently returned `0`, hiding real failures.	Caught the specific `sqlite3.Error` and logged it with `logger.exception(...)` before returning `0`, so failures are visible and only the intended error class is handled.	**Fixed**
`assert` used for a runtime check	**Security/ Reliability** — Bandit **B101** (Low), CWE-703	38	`assert feeCollected("R1") == 3600` guards a business invariant, but asserts are stripped when Python runs optimised (`-O`), so the check silently disappears in production.	Replaced with an explicit `if ... != 3600: raise RuntimeError(...)`, which always executes regardless of optimisation flags.	**Fixed**
Unused function argument	**Code quality** — Pylint **W0613**	13	`routeRoster(route, driver)` accepted a `driver` parameter it never used (callers even passed `Non`	Removed the `driver` parameter (`route_roster(route)`) and updated every call site.	**Fixed**

Issue	Type/Tool	Line(s)	Description	Fix Approach	Status
			`e/"Mr. Das"`). Dead parameter — confusing API.		
File opened without `with` / no encoding	**Code quality** — Pylint **R1732**, **W1514**	18	`exportRoster` used `open(...)/out.close()` manually; an exception mid-write would leak the file handle, and the text encoding was platform-dependent.	Used a context manager `with open(path, "w", encoding="utf-8") as out:` so the file is always closed and the encoding is explicit.	**Fixed**
Non-PEP8 function names (camelCase)	**Style** — Pylint **C0103**, Flake8 naming	8,13,17,23,27	Functions `enrollRider`, `routeRoster`, `exportRoster`, `dispatchAlert`, `feeCollected` used camelCase instead of PEP 8 `snake_case`.	Renamed to `enrol_rider`, `route_roster`, `export_roster`, `dispatch_alert`, `fee_collected` and updated all calls.	**Fixed**
Missing docstrings	**Style** — Pylint **C0114**, **C0116**	1,8,13,17,23,27,34	Module and every function lacked a docstring.	Added a module docstring and one-line docstrings for each function.	**Fixed**
Lines too long	**Style** — Pylint **C0301**, Flake8 **E501**	5,9,14,29	Several statements exceeded the line-length limit (up to 116 chars).	Wrapped long SQL/`CREATE` statements into implicit-concatenated literals and split call arguments across lines.	**Fixed**
Missing whitespace after commas	**Style** — Flake8 **E231**	8	`def enrollRider(student,route,stop,fee):` had no space after commas.	Reformatted the signature with spaces (part of the rename/reformat).	**Fixed**
Blank-line before top-level `def`	**Style** — Flake8 (E302 family)	23	Inconsistent blank lines between functions in the original.	Normalised to two blank lines between top-level definitions.	**Fixed**
`subprocess` import flagged	**Security (advisory)** — Bandit **B404** (Low)	1 (now 9)	Bandit notes that importing `subprocess` *can* be risky. It is an informational blacklist, not an exploit.	**Kept** — the dispatch alert legitimately shells out to `echo`; the actual risk (B602) is removed. Removing `subprocess` would delete intended functionality just to silence a Low advisory.	**Remaining (justified)**
`subprocess` call + partial path	**Security (advisory)** — Bandit **B603**, **B607** (Low)	50	After the fix, Bandit still emits Low reminders: B603 "check untrusted input" and B607 "partial executable path" (`echo` has no directory).	**Kept** — these are inherent Low-severity notices for any safe `subprocess` use. `route` is passed as a single argv element (never reinterpreted by a shell), so it cannot inject commands. Hardcoding `/bin/echo` would be less portable for no real security gain.	**Remaining (justified)**

"Fixed" = resolved and re-verified with the tools; "Remaining (justified)" = a Low-severity advisory documented rather than suppressed. Original line numbers refer to `school_transport_original.py`.

5. Key Code Fixes — Before / After

Snippets are quoted verbatim from `school_transport_original.py` (before) and the fixed `school_transport.py` (after).

SQL injection via f-string (Bandit B608 — Medium, CWE-89)

BEFORE — ORIGINAL

```
def enrollRider(student,route,stop,fee):
        conn.execute(f"INSERT INTO riders (student, route, stop, fee)
        VALUES ('{student}', '{route}', '{stop}', {fee})")
    conn.commit()
```

AFTER — FIXED

```
def enrol_rider(student, route, stop, fee):
    """Insert a rider using a parameterised query."""
    conn.execute(
        "INSERT INTO riders (student, route, stop, fee) "
        "VALUES (?, ?, ?, ?)",
        (student, route, stop, fee),
    )
    conn.commit()
```

Fix: Values are bound with ? placeholders, so they can no longer be parsed as SQL.

OS command injection via shell=True (Bandit B602 — High, CWE-78)

BEFORE — ORIGINAL

```
def dispatchAlert(route):
        subprocess.run("echo dispatching bus " + route, shell=True,
        check=False)
```

AFTER — FIXED

```
def dispatch_alert(route):
    """Send a dispatch message without invoking a shell."""
    subprocess.run(
        ["echo", "dispatching bus", route],
        check=False,
    )
```

Fix: shell=True removed; the route is passed as a single argv element, never to a shell.

Overly broad exception handler (Pylint W0702 / Flake8 E722)

BEFORE — ORIGINAL

```
def feeCollected(route):
    try:
                return conn.execute("SELECT SUM(fee) FROM riders WHERE
            route = ?", (route,)).fetchone()[0]
    except:
        return 0
```

AFTER — FIXED

```
def fee_collected(route):
    """Return the total fee for a route, or 0 if the query fails."""
    try:
        row = conn.execute(
            "SELECT SUM(fee) FROM riders WHERE route = ?",
            (route,),
        ).fetchone()
        return row[0]
    except sqlite3.Error:
        logger.exception("could not total fees for route %s", route)
        return 0
```

Fix: Catch the specific sqlite3.Error and log it, instead of silently swallowing everything.

assert used for a runtime invariant (Bandit B101 — Low, CWE-703)

BEFORE — ORIGINAL

```
    assert feeCollected("R1") == 3600
```

AFTER — FIXED

```
    if fee_collected("R1") != 3600:
        raise RuntimeError("R1 fee total mismatch")
```

Fix: Explicit if/raise is always executed, even under python -O where asserts are stripped.

Resource handling: file open without with / encoding (Pylint R1732, W1514)

BEFORE — ORIGINAL

```
def exportRoster(path, route):
    out = open(path, "w")
    for name in routeRoster(route, None):
        out.write(name + "\n")
    out.close()
```

AFTER — FIXED

```
def export_roster(path, route):
    """Write a route roster to a text file, one name per line."""
    with open(path, "w", encoding="utf-8") as out:
        for name in route_roster(route):
            out.write(name + "\n")
```

Fix: A with-block guarantees the file is closed; UTF-8 encoding is now explicit.

In addition, every function was renamed to `snake_case`, a module docstring and per-function docstrings were added, over-long lines were wrapped, the unused `driver` parameter was removed, and the bare top-level `main()` call was moved under `if __name__ == "__main__":`.

6. Original vs Final Analysis Results

Tool	Original	Final
Pylint (score)	4.00/10	**10.00/10**
Bandit — High	1	**0**
Bandit — Medium	1	**0**
Bandit — Low	2	3 (advisory, see §8)
Flake8 — findings	9	**0** (clean)

> **Behaviour preserved.** Running the fixed program produces output identical to the original (`diff` of stdout and the generated `route_r1.txt` is empty): the dispatch alert `dispatching bus R1`, the roster `['Zoya']`, and the roster file containing `Tara` / `Kabir`.

Final tool verdicts (verbatim)

```
Your code has been rated at 10.00/10
```

```
Result : clean — 0 findings (original file had 9 findings; see
    flake8_report_original.txt)
```

7. Reflection Answers

1. What is static code analysis, and how does it differ from dynamic testing?

Static code analysis inspects the **source code itself** without executing the program. Tools parse the code (often into an abstract syntax tree) and apply rules to flag bugs, style violations, and security patterns. In this lab, Pylint, Bandit and Flake8 all examined `school_transport.py` without the register ever running.

Dynamic testing, by contrast, runs the program with inputs and observes its behaviour at runtime (unit tests, integration tests, manual execution). It can catch logic errors that only surface on a particular input path, but it can only find problems in code that the tests actually reach. They are complementary: static analysis finds problems across *all* code paths up front; dynamic testing confirms the code behaves correctly on the paths you exercise. I

used both here — the three static tools to find the issues, and running the program before and after to confirm behaviour was preserved.

2. What types of issues did Pylint, Bandit, and Flake8 identify?

- **Pylint** found a mix of correctness and convention issues on the original (score 4.00/10): a bare `except` (`W0702`), an unused `driver` argument (`W0613`), opening a file without `with` / encoding (`R1732`, `W1514`), camelCase function names that violate PEP 8 (`C0103`), missing module and function docstrings (`C0114`, `C0116`), and over-long lines (`C0301`).
- **Bandit** found the security problems: **B602** *High* — `shell=True` in `subprocess.run` (OS-command injection), **B608** *Medium* — an `INSERT` built by f-string (SQL injection), **B101** *Low* — `assert` used for a runtime check, plus **B404** *Low* — the `subprocess` import itself.
- **Flake8** found pure PEP 8 style issues: over-long lines (`E501`), missing whitespace after commas (`E231`), and the bare `except` (`E722`).

So the three tools covered different layers: Bandit = security, Flake8 = style, Pylint = a blend of correctness and convention.

3. Which issues were the most important to fix, and why?

The **security findings mattered most**, because they could let an attacker run arbitrary commands or manipulate the database — the kind of defect that a functional test would never reveal on normal data.

- **B602 `shell=True` (High)** was the worst: `dispatch_alert(route)` concatenated user-controlled `route` into a shell command, so a value like `R1; rm -rf /` or `` R1`malicious` `` would execute directly. This is a real, exploitable command-injection vector (CWE-78).
- **B608 SQL injection (Medium)**: building the `INSERT` with an f-string means a name containing a single quote (a perfectly plausible student name like `O'Brien`) could break the statement or inject SQL (CWE-89).
- **The bare `except:` (W0702/E722)** was the most important *reliability* fix: it silently swallowed every error and returned `0`, so a database failure would masquerade as "no fees collected" instead of being reported.
- **`assert` for a business check (B101)** was subtle but real: optimised bytecode (`python -O`) strips asserts, so the fee-total check would quietly vanish in production.

4. How did the fixes improve the code's quality, security, readability, or maintainability?

- **Security:** parameterised queries (`?` placeholders) and an argv list without a shell eliminate both injection vectors — untrusted data is now *data*, never parsed as code. Replacing the assert with an explicit `raise` keeps the invariant enforced even under `-O`. Bandit went from 1 High + 1 Medium to 0 High + 0 Medium.
- **Quality/reliability:** catching `sqlite3.Error` (instead of every exception) and logging it means real failures are surfaced rather than hidden, while still returning a safe default.
- **Readability/maintainability:** renaming to `snake_case`, adding module and function docstrings, wrapping long lines, and using `with open(...)` brought Pylint from 4.00/10 to 10.00/10 and Flake8 to a clean run. Removing the dead `driver` parameter made the roster function's contract honest. None of these changed what the program *does* — its output is byte-for-byte the same — they only made the code safer and clearer.

5. What limitations does static analysis have?

- **It cannot see runtime behaviour.** It does not know that `fee_collected("R1")` actually returns `3600`, or that the register's logic is correct — only dynamic testing confirms that.
- **False positives / noise.** Bandit's **B404** simply flags *importing* `subprocess`, and **B603**/*B607* remain even though my call is safe (no shell, single argv element). These are advisories, not defects; blindly "fixing" them would mean deleting intended functionality.
- **False negatives.** Static tools miss issues that depend on data flow, configuration, or environment (race conditions, memory growth, logic bugs) and cannot reason about every path.
- **Rule/toolkit dependence.** Flake8's default line-length (79) differs from Pylint's (100), so the same file scores differently per tool; coverage depends on the rules you enable. It enforces *conventions*, not *correctness*.

6. How can static analysis contribute to software quality in a development team?

Run in CI/CD, static analysis gives **fast, cheap, objective feedback on every commit** — catching an injection or a bare `except` at review time rather than in production, where such bugs are expensive and dangerous. It enforces a consistent style automatically (naming, docstrings, line length), so code reviews focus on logic instead of formatting, and new team members inherit house standards without manual policing. Because tools like Bandit map findings to CWEs, they raise developer security awareness over time. The key is **triage, not zero-warning dogma**: team leads should treat findings as signals — fixing the High/Medium security issues here, documenting the remaining Low advisories with justification rather than suppressing them — and pair static analysis with dynamic tests and review for real coverage.

8. Remaining Bandit Warnings & Justifications

After fixing every High and Medium finding, three **Low-severity advisories** remain. These are inherent to the legitimate, now-safe use of `subprocess` for the dispatch alert; removing the feature entirely would delete intended functionality just to silence a warning, which the lab handout discourages. They are documented, not suppressed.

Verbatim from the final Bandit report

```
>> Issue: [B404:blacklist] Consider possible security implications
    associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
      More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blackli
       st_imports.html#b404-import-subprocess
   Location: ./school_transport.py:9:0
8       import sqlite3
9       import subprocess
10

--------------------------------------------------
>> Issue: [B607:start_process_with_partial_path] Starting a process with a
    partial executable path
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
      More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b607_start
       _process_with_partial_path.html
   Location: ./school_transport.py:50:4
49          """Send a dispatch message without invoking a shell."""
50          subprocess.run(
51              ["echo", "dispatching bus", route],
52              check=False,
53          )
54

--------------------------------------------------
>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call -
    check for execution of untrusted input.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
```

```
      More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subpr
       ocess_without_shell_equals_true.html
   Location: ./school_transport.py:50:4
49          """Send a dispatch message without invoking a shell."""
50          subprocess.run(
51              ["echo", "dispatching bus", route],
52              check=False,
53          )
54

--------------------------------------------------
```

Advisory	Severity	Justification
B404 — `import subprocess` blacklist	Low	Informational note that importing `subprocess` can be risky. It is not an exploit; the actual risk (B602) is fixed.
B603 — subprocess call, check untrusted input	Low	A generic reminder that fires on *any* subprocess call. Here `route` is a single argv element, never re-parsed by a shell, so it cannot inject commands.
B607 — partial executable path (`echo`)	Low	`echo` has no directory component. Pinning `/bin/echo` would be less portable for no real security gain; the command is not shell-parsed.

9. Conclusion

Static analysis exposed issues across security, code-quality and style. Eleven were resolved — including the two exploitable injection vectors and the error-swallowing bare `except` — clearing all High and Medium Bandit findings and reaching **Pylint 10.00/10** and a clean Flake8 run, with program behaviour unchanged. Three Low advisories remain and are justified as false-positive-style notices on a now-safe call. The lab demonstrates static analysis's core value: objective, pre-execution detection of defects — especially security flaws — that dynamic testing on normal inputs would miss.